Shraeyaa Dhaigude
D20A - 22

Experiment.4

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Theory:

Solidity and Smart Contract Fundamentals
1. Primitive Data Types, Variables, and Functions (pure, view)

Solidity is a high-level, statically typed programming language used for developing smart contracts on the Ethereum blockchain. Primitive data types serve as the fundamental building blocks of Solidity and play a crucial role in defining variables, performing computations, and managing data within a contract.

Commonly used primitive data types include:

- **uint / int**: Used to represent unsigned and signed integers of different sizes, such as uint256 or int128. These are widely used for financial calculations, counters, and token balances.
- **bool**: Represents logical values (true or false) and is commonly used in decision-making statements such as access control and validations.
- **address**: Stores a 20-byte Ethereum account or contract address, which is essential for identifying users and interacting with other contracts.
- **bytes / string**: Used for storing binary and textual data, respectively, such as user names or encrypted information.

Variables in Solidity are categorized into three types based on their scope and lifetime:

- **State Variables**: Stored permanently on the blockchain and consume gas when modified. Example: storing the contract owner's address.
- **Local Variables**: Temporary variables that exist only during function execution and are not stored on-chain.
- **Global Variables**: Predefined variables such as msg.sender, msg.value, and block.timestamp, which provide information about the transaction and blockchain environment.

Functions in Solidity define the logic and behavior of smart contracts. Two important function types include:

- **pure**: Functions that neither read nor modify blockchain state; they operate only on input values. Example: a function that adds two numbers.
- **view**: Functions that can read state variables but cannot modify them. Example: a function that returns an account balance.

These function types help in optimizing gas usage and ensuring secure and predictable contract execution.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input parameters and return output values, enabling interaction between users and smart contracts. Inputs allow external users or other contracts to pass data into a function, while outputs enable the function to return computed results.

For example, a function may accept an amount of Ether as input and return a boolean value indicating whether a transaction was successful. Solidity also supports named return variables, which enhance code readability and simplify debugging.

3. Visibility, Modifiers, and Constructors

Function visibility defines who can access a particular function within or outside the contract:

- **public**: Accessible both internally and externally.
- **private**: Accessible only within the same contract.
- **internal**: Accessible within the contract and its derived contracts.
- **external**: Can only be called by external accounts or other contracts.

Modifiers are special functions that modify the behavior of other functions. They are commonly used for security and access control. For instance, an onlyOwner modifier can restrict certain functions so that only the contract owner can execute them.

A **constructor** is a special function that executes only once during contract deployment. It is primarily used to initialize important variables, such as assigning the deploying account as the contract owner.

4. Control Flow: if-else and Loops

Solidity supports standard control flow structures similar to traditional programming languages.

- **if-else statements** allow conditional decision-making in contract logic. For example, a contract can verify whether a user has sufficient balance before processing a transaction.

- **Loops (for, while, do-while)** enable repeated execution of code, such as iterating through an array of registered users. However, loops must be used carefully, as excessive iterations increase gas consumption and may make transactions costly or even fail due to gas limits.

5. Data Structures: Arrays, Mappings, Structs, and Enums

Solidity provides several data structures to efficiently manage and organize data within smart contracts.

- **Arrays**: Used to store ordered collections of elements. They can be fixed-size or dynamic. Example: an array storing addresses of registered users.
- **Mappings**: Key-value storage structures that allow fast data retrieval. Example: mapping(address => uint) to store user balances. Unlike arrays, mappings do not support iteration.

Structs: Allow grouping of related data into a single entity. Example:

```
struct Player {
string name;
uint score; }
```

- This structure helps store multiple attributes of a player in an organized manner.

Enums: Used to define a set of predefined constants, improving code clarity and readability. Example:

```
enum Status { Pending, Active, Closed }
```

6. Data Locations

Solidity defines three primary data locations that determine where variables are stored and how they behave in memory:

- storage: Permanent data stored on the blockchain. Used for state variables.
- memory: Temporary data that exists only during function execution. Used for local variables and function inputs.
- calldata: A non-modifiable and non-persistent location used for external function parameters. It is more gas-efficient than memory.

Understanding data locations is essential because they directly impact gas costs, efficiency, and contract performance.

7. Transactions: Ether, Wei, Gas, and Sending Transactions

Ether and Wei:
Ether is the native cryptocurrency of the Ethereum blockchain. The smallest unit of Ether is Wei, where:
1 Ether = 10^{18} Wei.
This ensures high precision in financial and smart contract transactions.

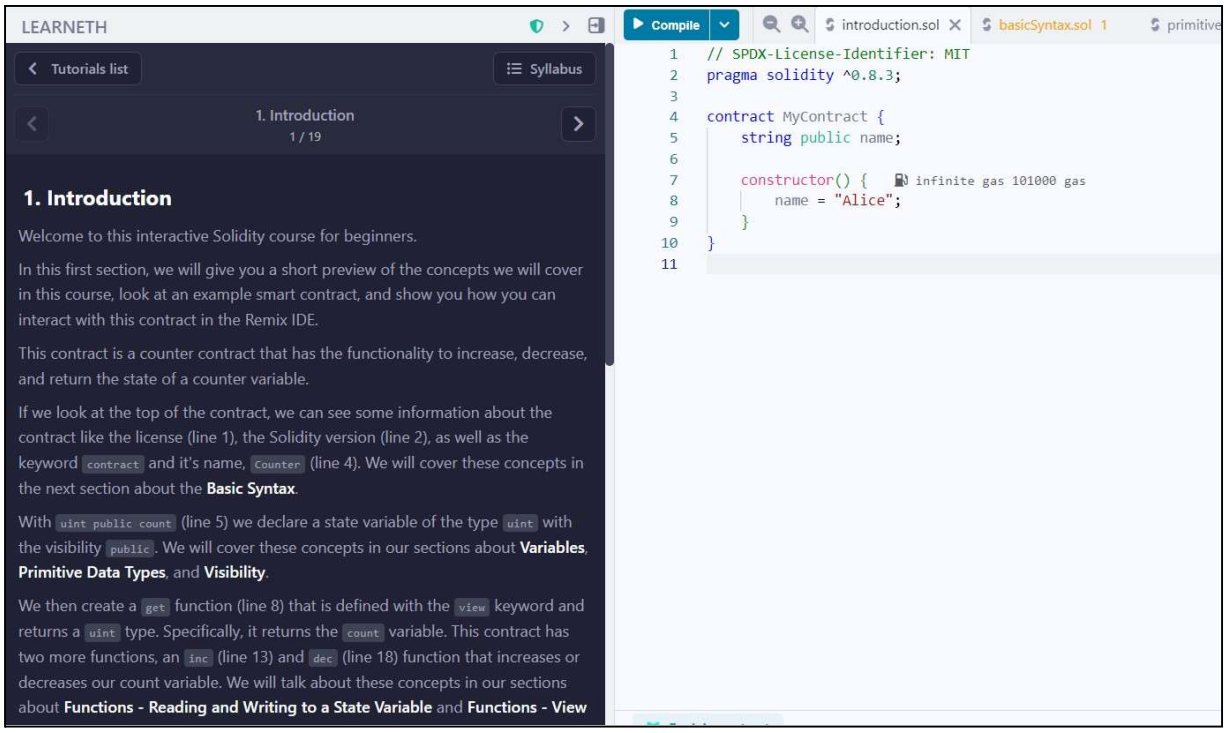
Gas and Gas Price:
Every transaction on Ethereum consumes gas, which represents the computational cost of executing operations. The gas price determines how much Ether is paid per unit of gas. A higher gas price results in faster transaction processing.

Sending Transactions:
Transactions are used to transfer Ether or interact with smart contracts. Common methods include:

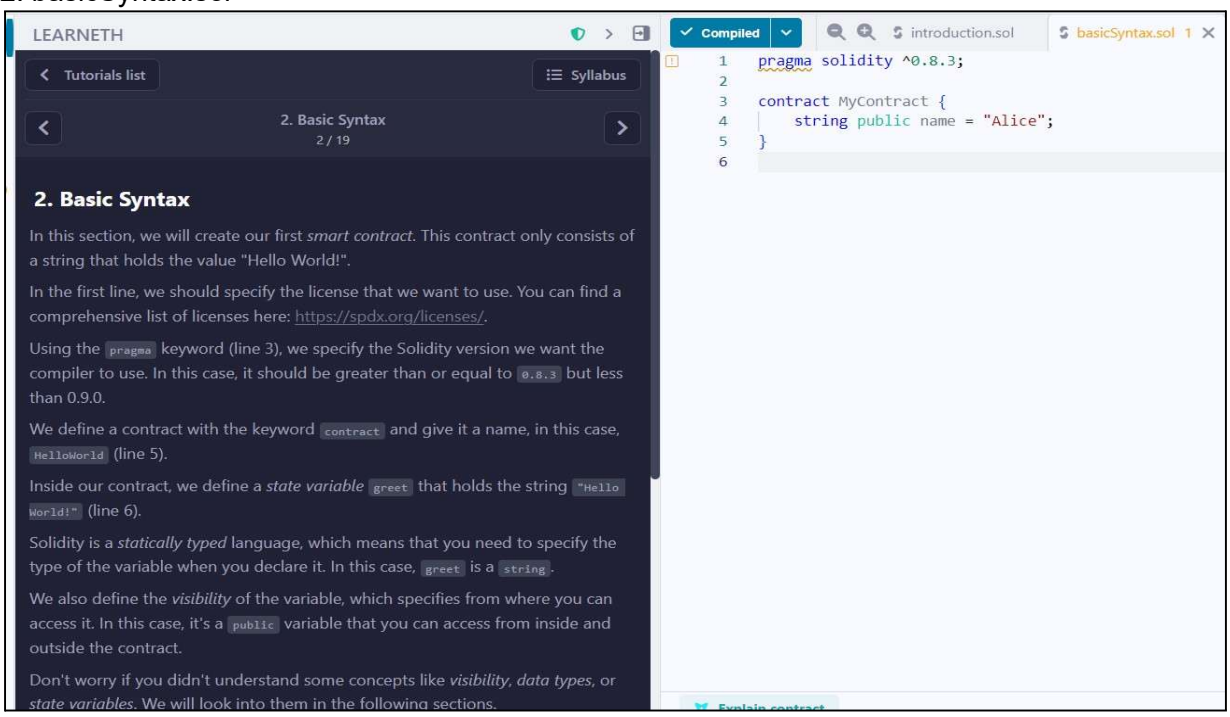
- transfer() – Safe but limited in gas usage.
- send() – Returns a boolean value indicating success or failure.
- call() – More flexible and recommended for modern Solidity development.

Efficient contract design is necessary to minimize gas consumption and reduce transaction costs.
Output -

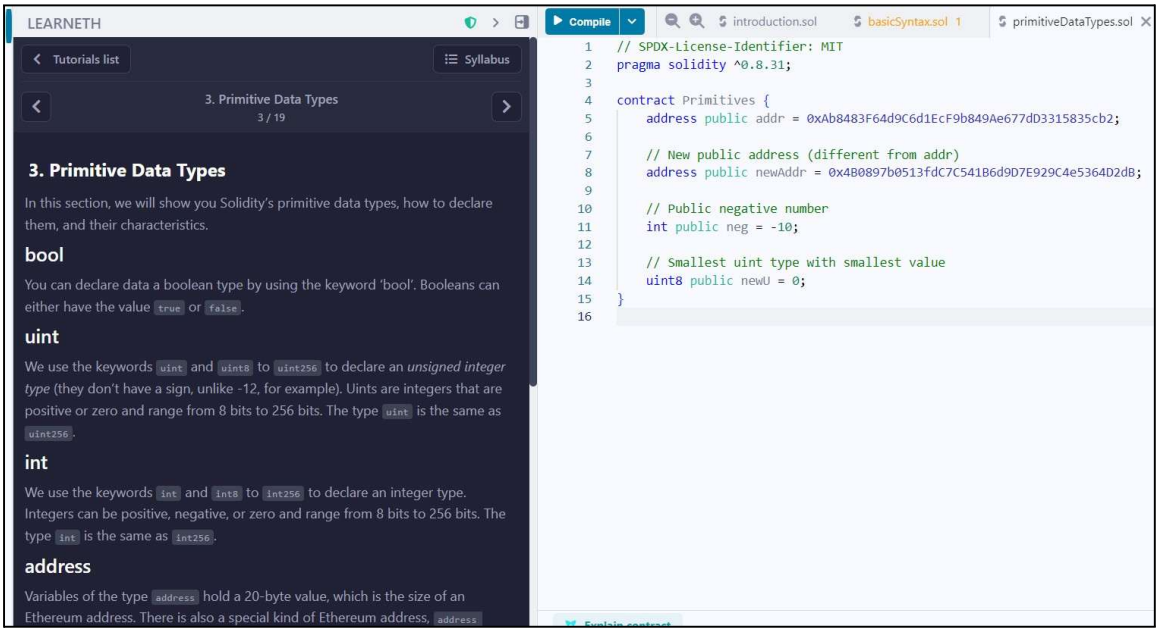
1.introduction.sol



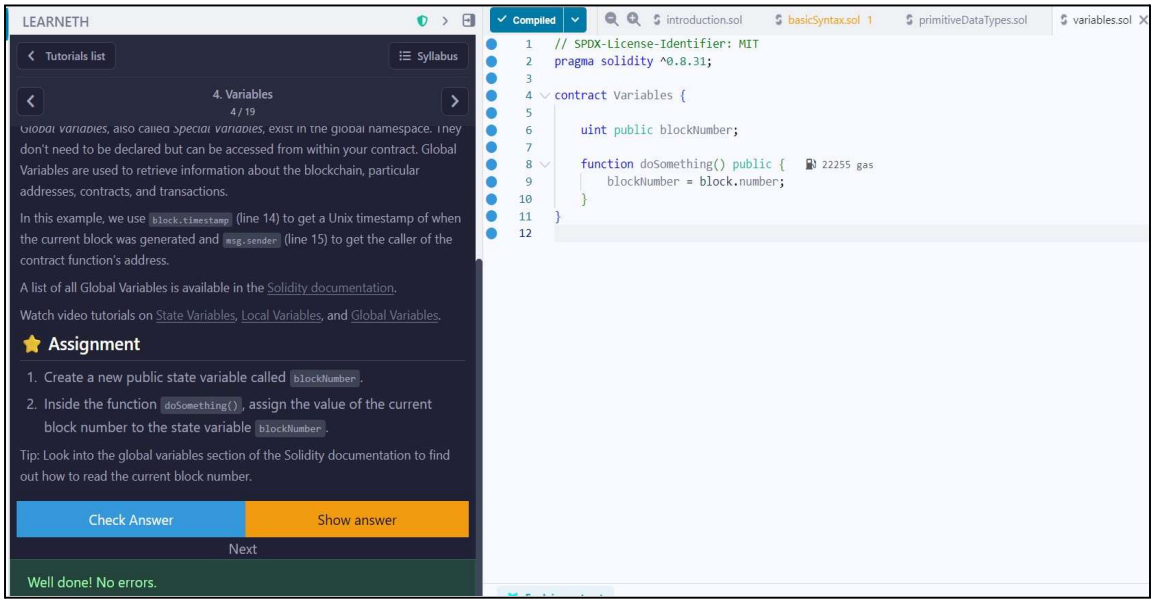
2. basicSyntax.sol



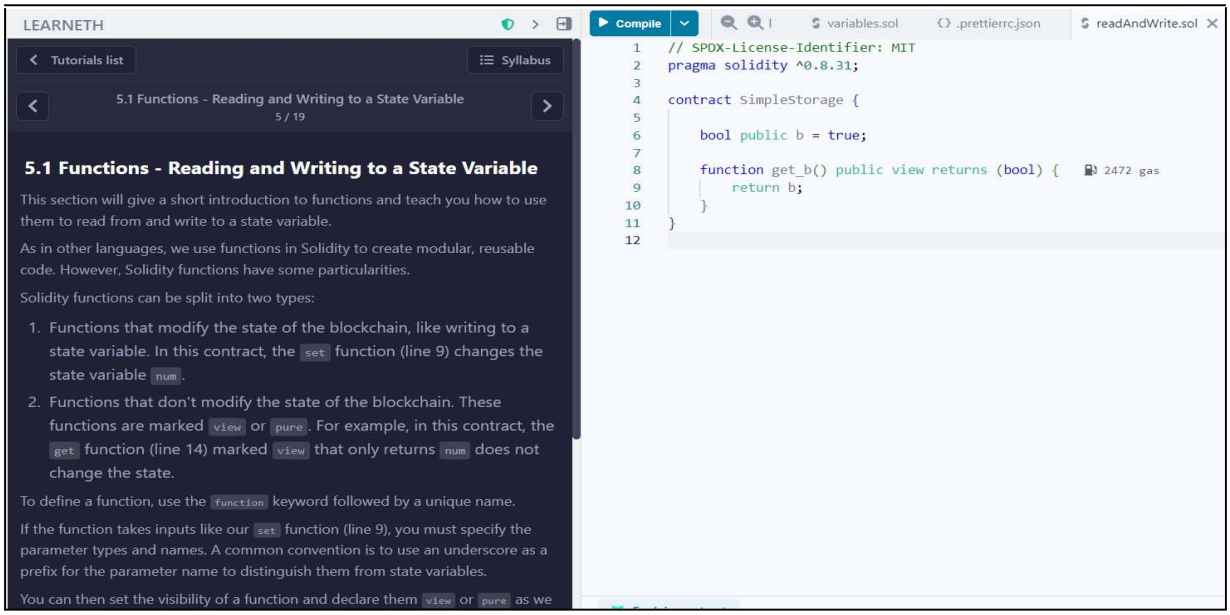
3. primitiveDataTypes.sol



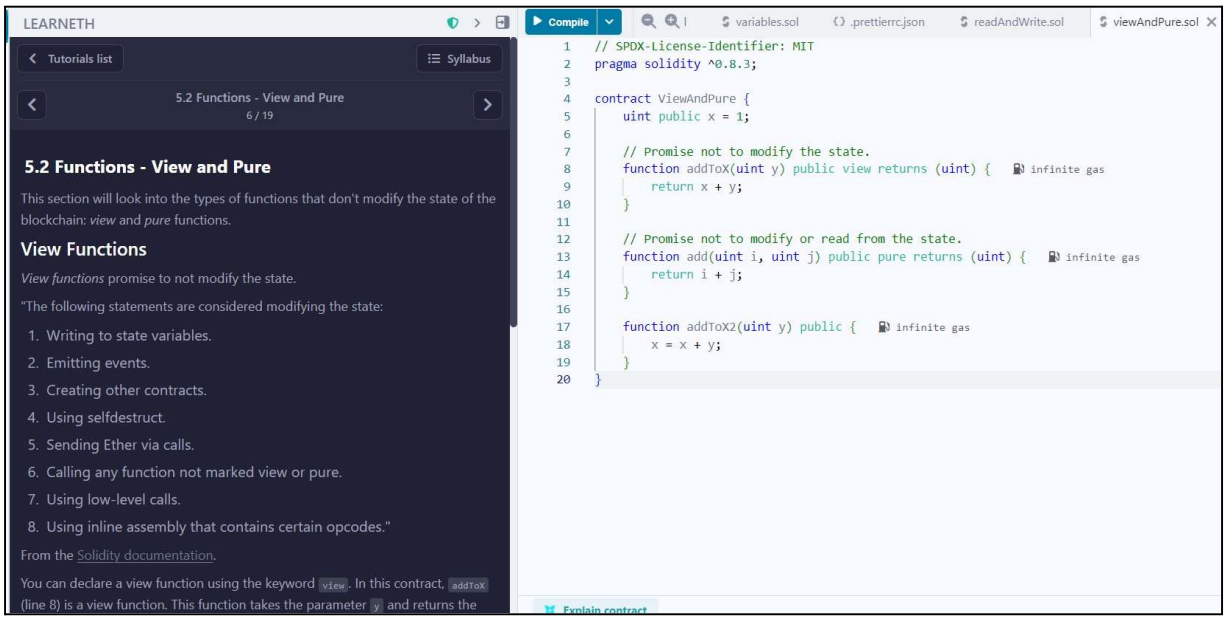
4. variables.sol



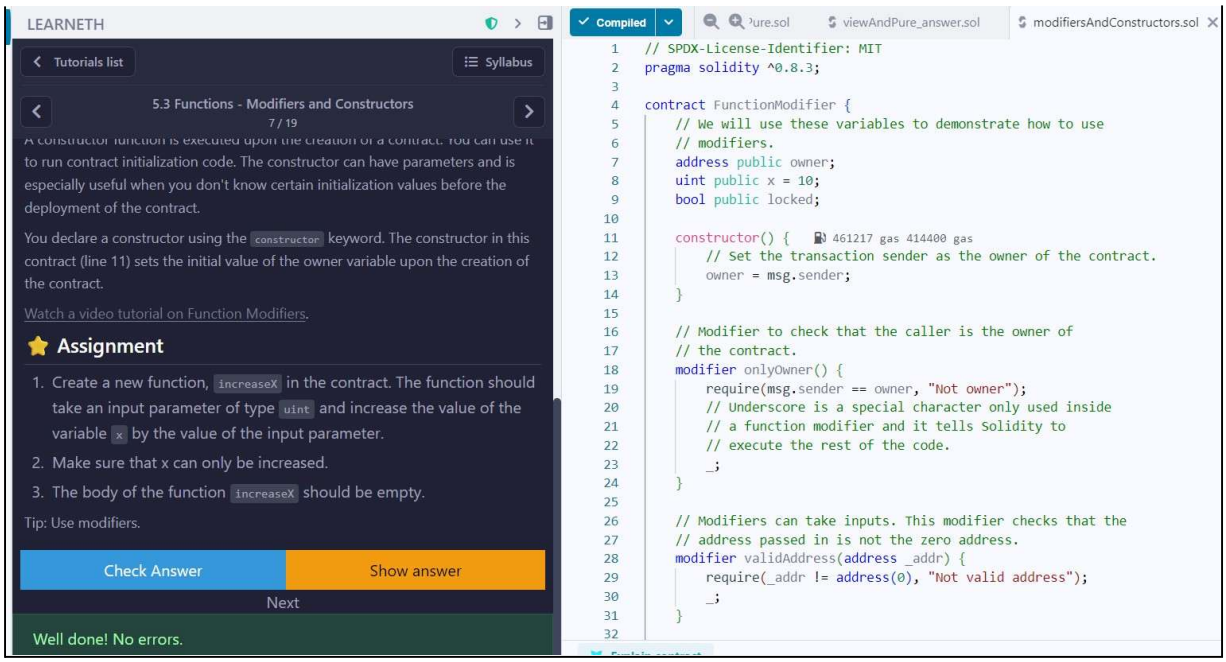
5.1 readAndWrite.sol



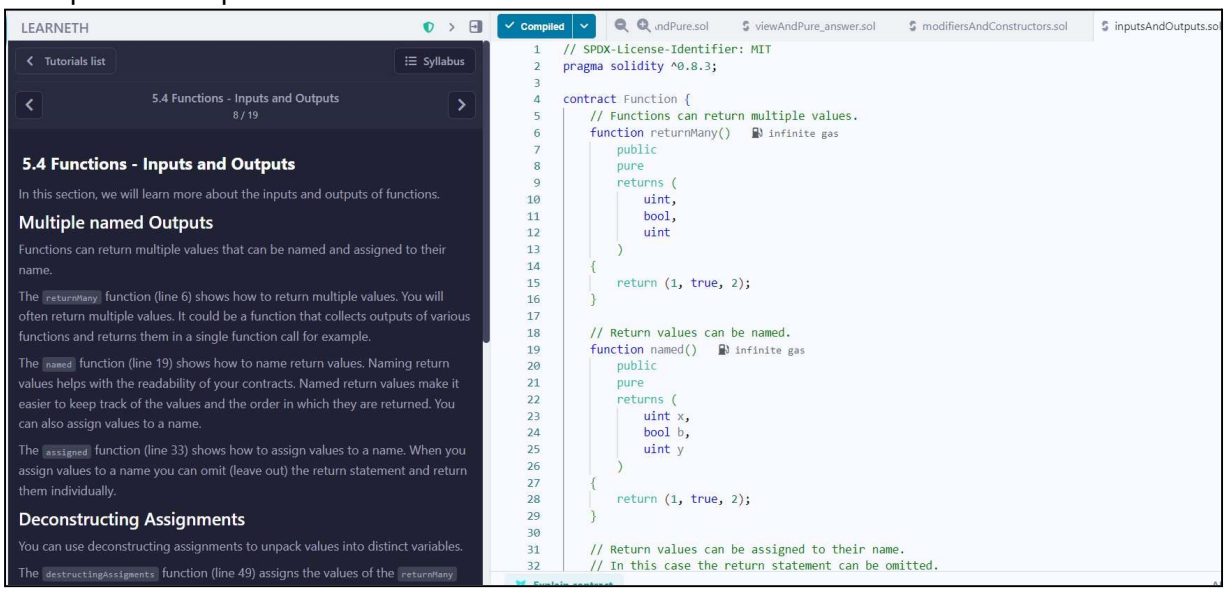
5.2 viewAndPure.sol



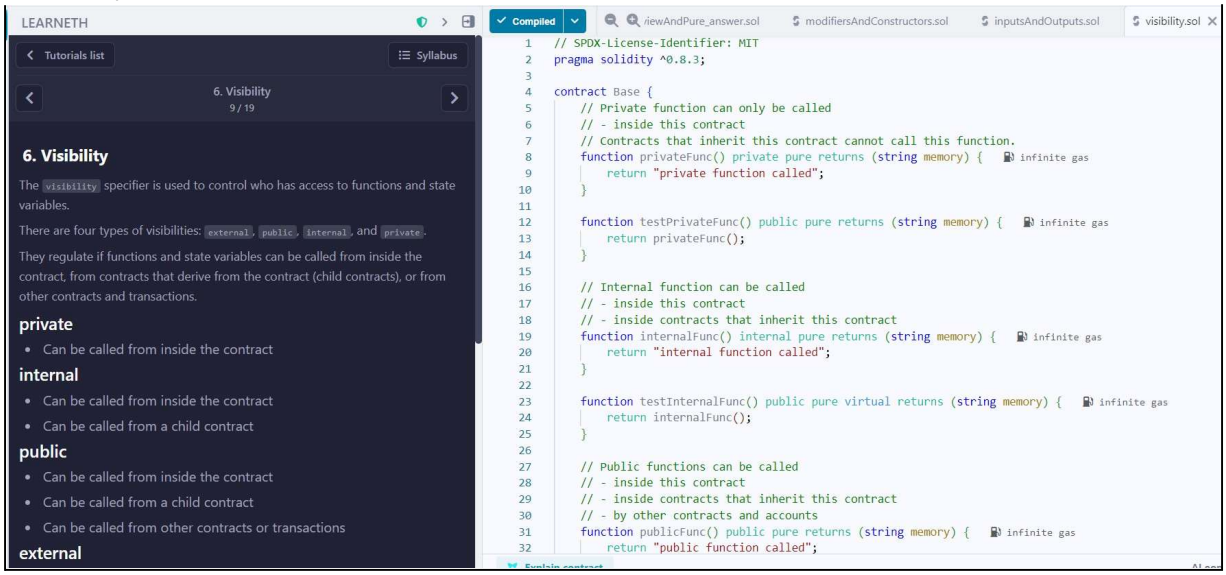
5.3 modifiersAndConstructor.sol



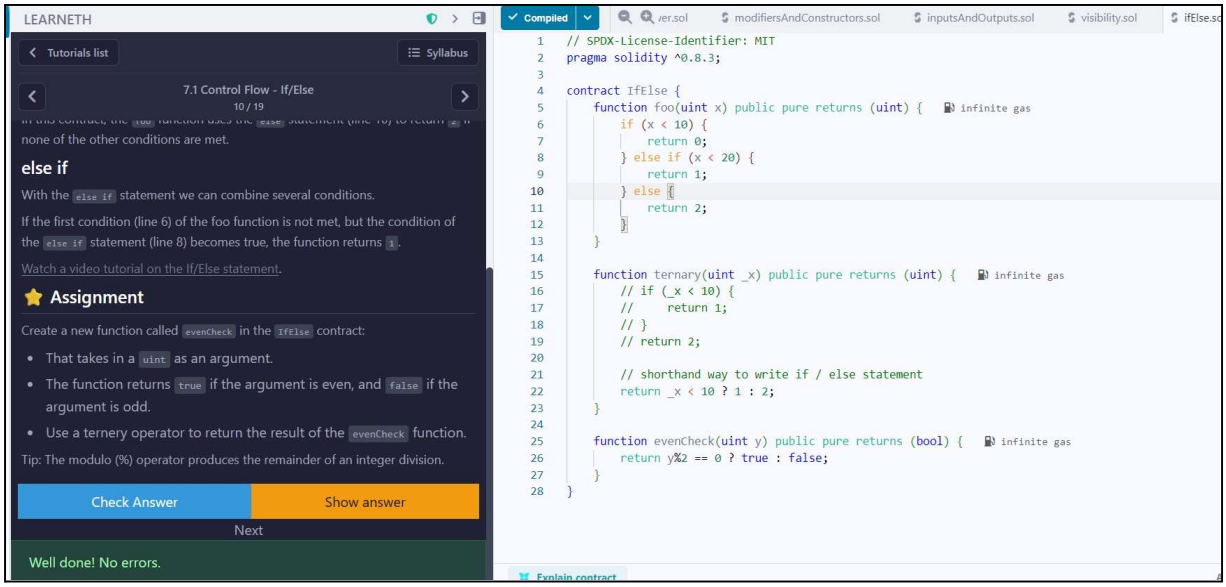
5.4 inputsAndOutputs.sol



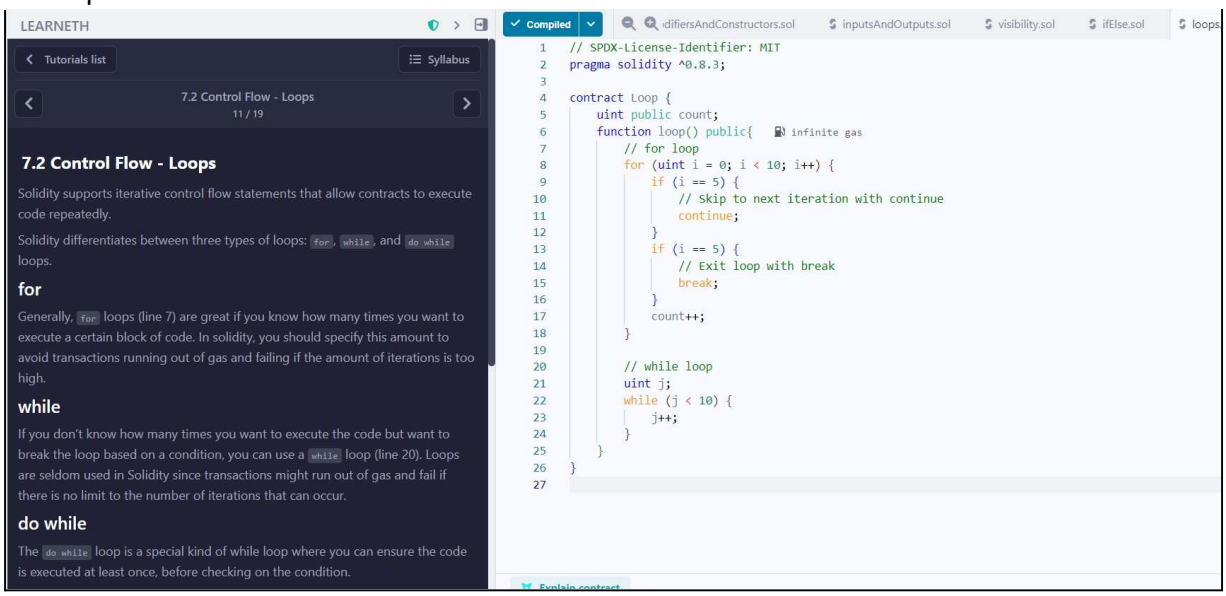
6. visibility.sol



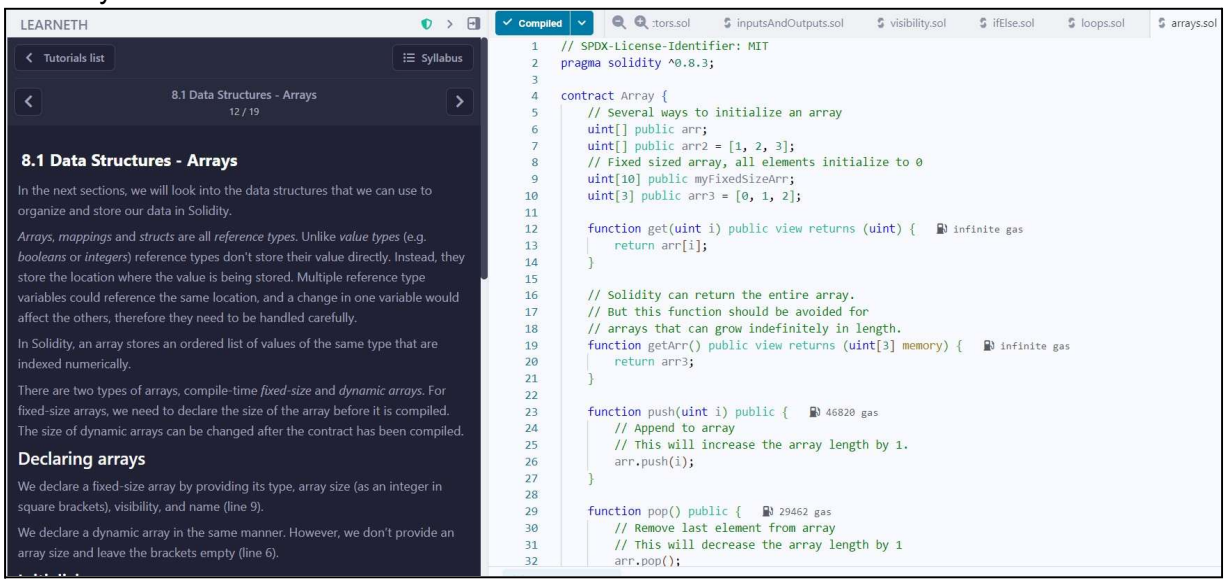
7.1 ControlFlow.sol



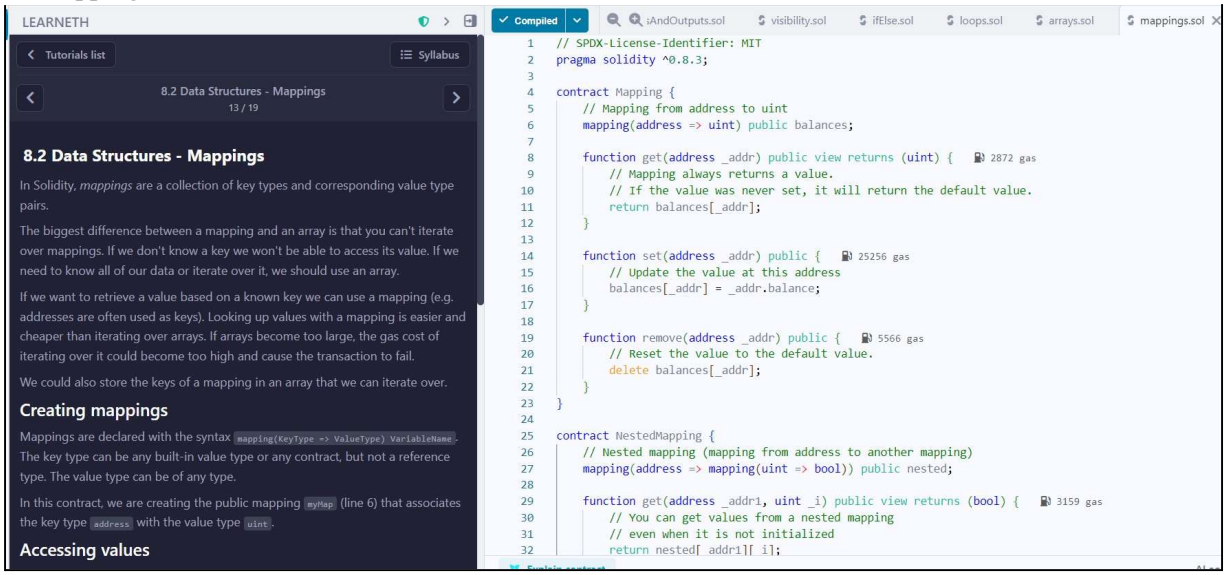
7.2 loops.sol



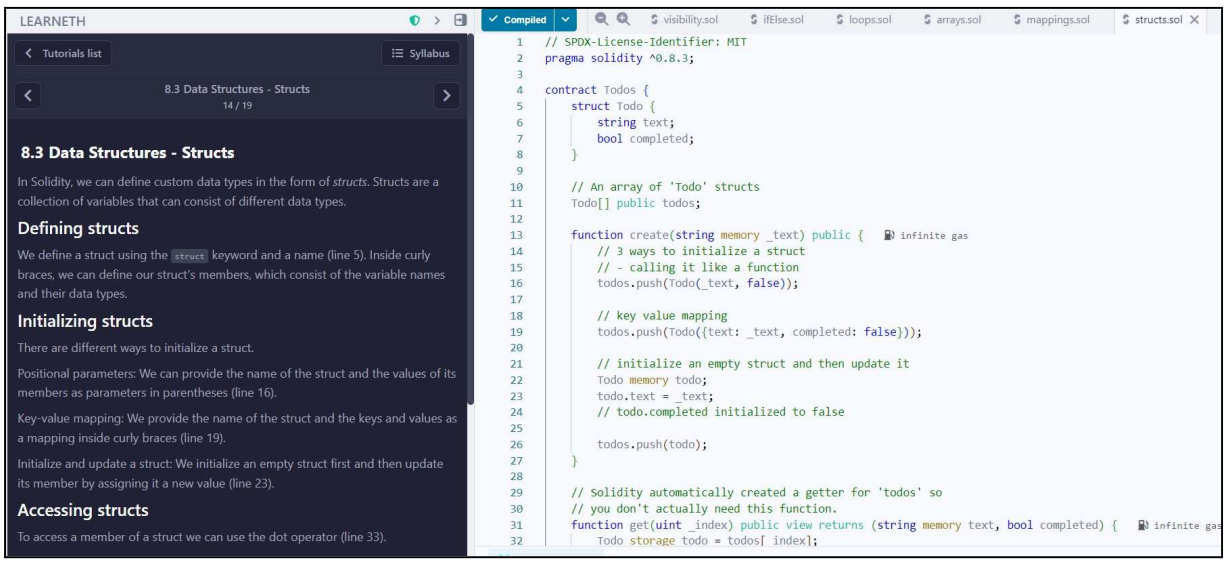
8.1 arrays.sol



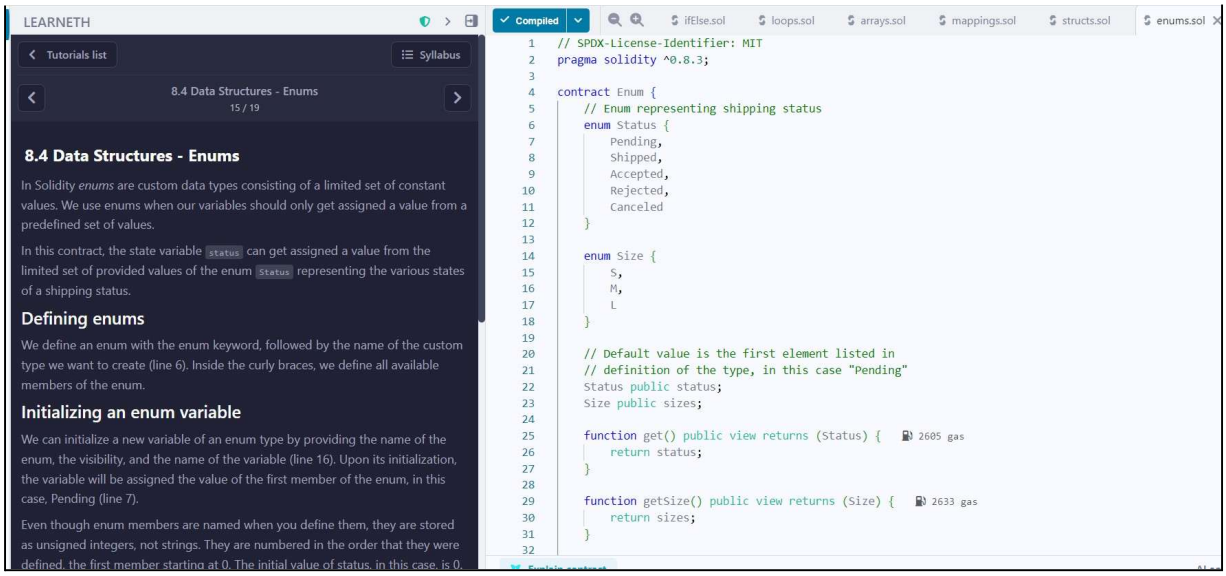
8.2 mapping.sol



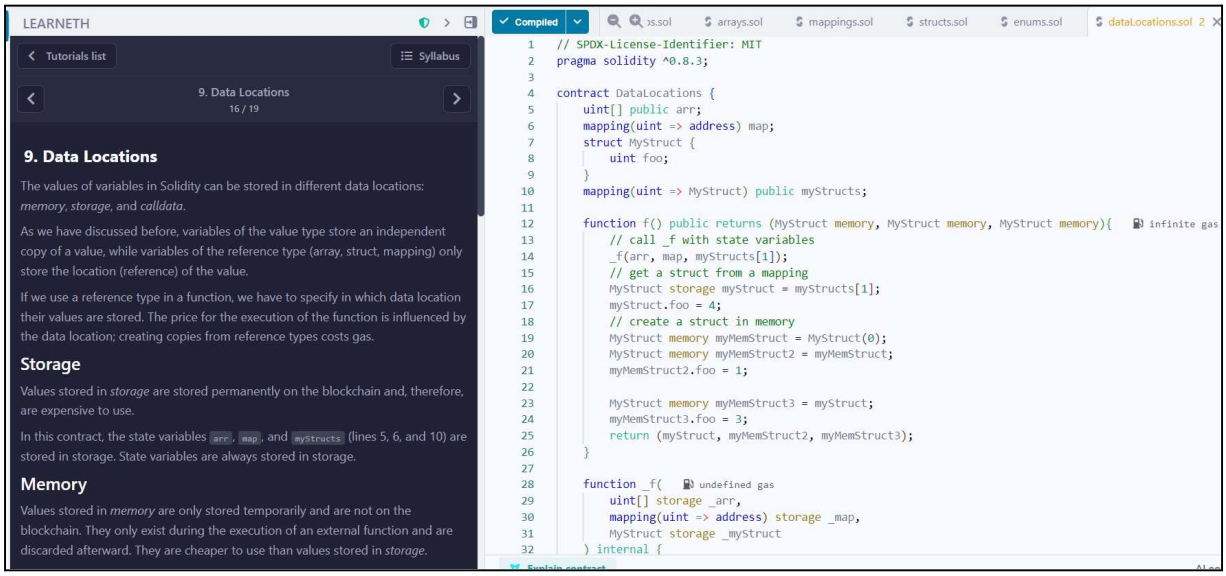
8.3 structs.sol



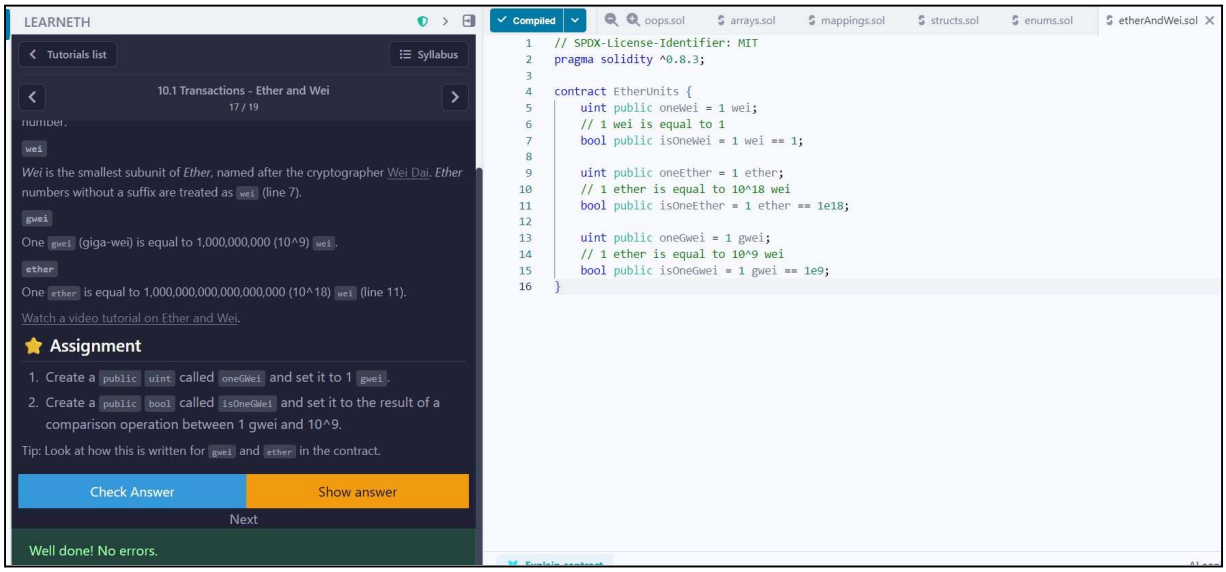
8.4 enums.sol


LEARNETH
Tutorials list
Syllabus
8.4 Data Structures - Enums
15 / 19
8.4 Data Structures - Enums
In Solidity enums are custom data types consisting of a limited set of constant values. We use enums when our variables should only get assigned a value from a predefined set of values.
In this contract, the state variable status can get assigned a value from the limited set of provided values of the enum Status representing the various states of a shipping status.
Defining enums
We define an enum with the enum keyword, followed by the name of the custom type we want to create (line 6). Inside the curly braces, we define all available members of the enum.
Initializing an enum variable
We can initialize a new variable of an enum type by providing the name of the enum, the visibility, and the name of the variable (line 16). Upon its initialization, the variable will be assigned the value of the first member of the enum, in this case, Pending (line 7).
Even though enum members are named when you define them, they are stored as unsigned integers, not strings. They are numbered in the order that they were
Compiled
ifElse.sol
loops.sol
arrays.sol
mappings.sol
structs.sol
enums.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Enum {
// Enum representing shipping status
enum Status {
Pending,
Shipped,
Accepted,
Rejected,
Canceled
}
enum Size {
S,
M,
L
}
// Default value is the first element listed in
// definition of the type, in this case "Pending"
Status public status;
Size public sizes;
function get() public view returns (Status) {
2605 gas
return status;
}
function getSize() public view returns (Size) {
2633 gas
return sizes;
}

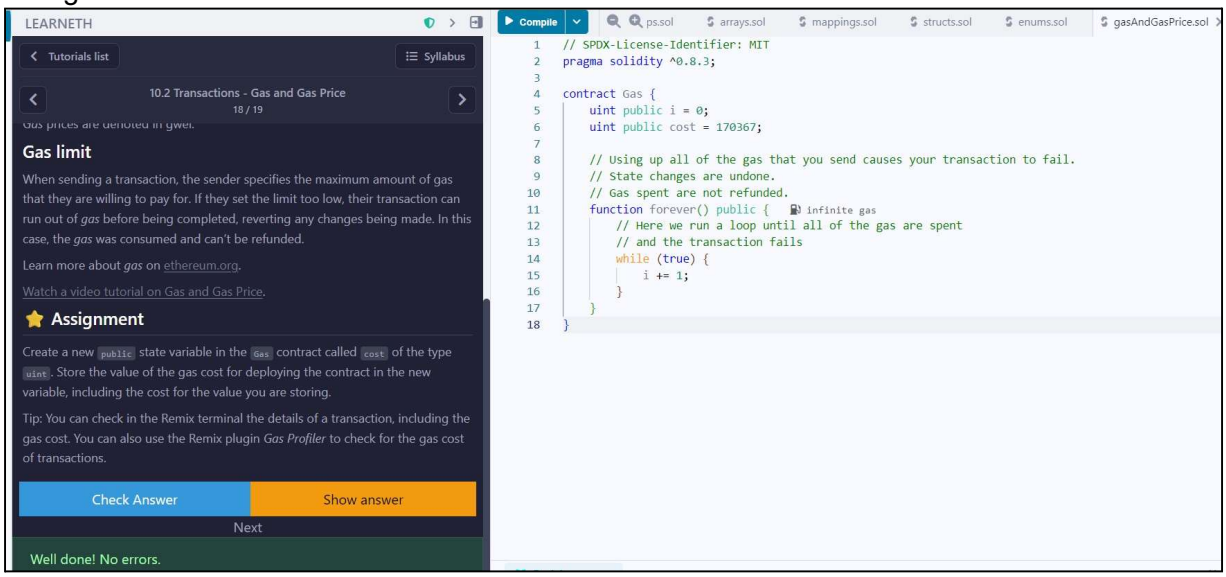
9. dataLocations.sol



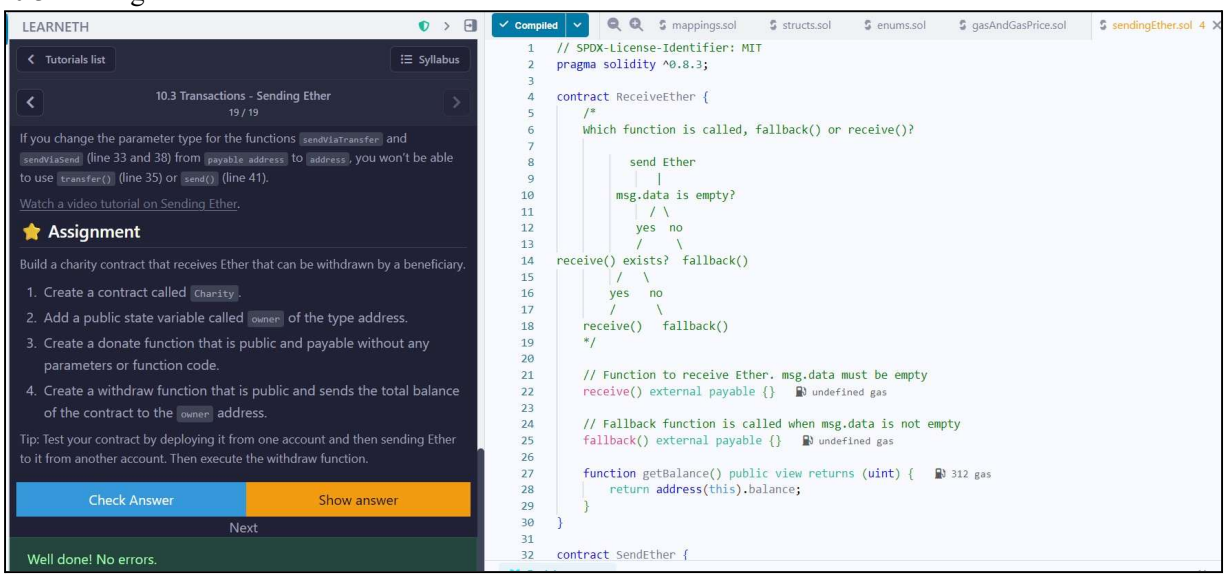
10.1 etherAndWei.sol



10.2 gasAndGasPrice.sol



10.3 sendingEther.sol



Conclusion - This experiment provides hands-on experience in Solidity programming by exploring core blockchain concepts, data types, control structures, and transaction handling. Understanding these fundamentals enables the design and deployment of secure, efficient, and reliable smart contracts on the Ethereum blockchain.